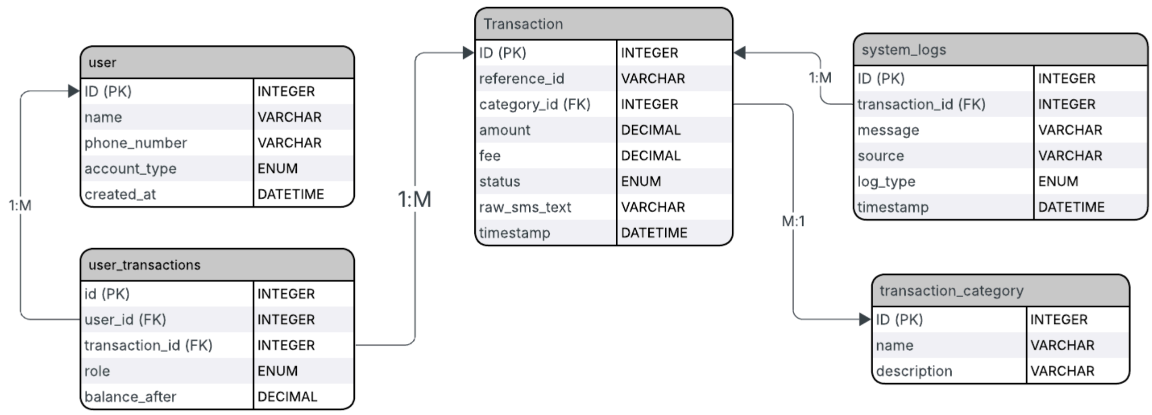
Team3 Database Design Documentation

We modeled MTN MoMo SMS data processing database around a transaction entity. The ***transactions*** table exists to hold the information we obtain from the parsed momo xml data in a clear and easy to query and understand structure.

MoMo SMS Data Processing System — ERD



Database Structure Explanation

1. **Transaction table:** As the core table, It holds the momo reference id, amount, fee, status, and other transaction related information. We also store the Raw momo SMS text, for data integrity and verification.
2. **User table:** Each phone number involved in a transaction is regarded as a user, and we save their unique phone numbers, account type, when they were added in the database, and name of the owner, or account name (example: MOMO code).
3. **User_Transactions table:** This is a junction table introduced to solve the Many-to-Many entity relationship issues e.g: one user participating in many transactions or one transaction involving

multiple users. It holds both user_id and transaction_id as foreign keys with multiple other attributes.

4. **System_logs table:** The sole role of the table is to store logs generated during operations of the system. Through tracking of transaction processing activities, errors, debugging information, etc. It holds transaction_id as a foreign key, id as the primary key and multiple other attributes.
5. **Transaction_category table:** The table is structurally made for saving transaction categories like deposit, withdrawal, transfer and bill payment. It contains id as its primary key and other attributes.

CRUD Operations Results

1. CREATE

when you run the ***database_setup.sql*** file, you get some sample data we have prepared.

You can run:

```
SELECT 'transaction_category' AS table_name, COUNT(*) AS records FROM transaction_category UNION ALL SELECT 'user', COUNT(*) FROM user UNION ALL SELECT 'transaction', COUNT(*) FROM transaction UNION ALL SELECT 'user_transactions', COUNT(*) FROM user_transactions UNION ALL SELECT 'system_logs', COUNT(*) FROM system_logs;
```

In your ***mysql*** console and get a list of your available records

```
+----------------------+---------+
| table_name           | records |
+----------------------+---------+
| transaction_category |       5 |
| user                 |       5 |
| transaction          |       5 |
| user_transactions    |       5 |
| system_logs          |       5 |
+----------------------+---------+
5 rows in set (0.00 sec)

mysql>
```

2. READ

You can run the read code blocks we have in ***crud_tests.sql*** and get a sample view of **READ** operations on the database.

```
mysql> SELECT t.reference_id, t.amount, t.fee, t.status, t.timestamp,
    ->        tc.name AS category
    -> FROM transaction t
    -> JOIN transaction_category tc ON t.category_id = tc.id;
+--------------+----------+--------+---------+---------------------+------------------+
| reference_id | amount   | fee    | status  | timestamp           | category         |
+--------------+----------+--------+---------+---------------------+------------------+
| TXN10001     |  5000.00 | 100.00 | SUCCESS | 2025-04-10 08:30:00 | P2P Transfer     |
| TXN10002     | 12500.00 |   0.00 | SUCCESS | 2025-07-22 09:15:00 | Merchant Payment |
| TXN10003     |  1000.00 |   0.00 | SUCCESS | 2025-09-05 10:00:00 | Airtime Purchase |
| TXN10004     | 50000.00 |   0.00 | SUCCESS | 2026-01-14 11:20:00 | Cash In          |
| TXN10005     | 15000.00 | 300.00 | FAILED  | 2026-03-30 12:05:00 | P2P Transfer     |
+--------------+----------+--------+---------+---------------------+------------------+
5 rows in set (0.00 sec)

mysql>
```

```
mysql> SELECT t.reference_id, t.amount, t.status,
    ->        u.name AS participant, u.phone_number,
    ->        ut.role, ut.balance_after
    -> FROM transaction t
    -> JOIN user_transactions ut ON t.id = ut.transaction_id
    -> JOIN user u ON ut.user_id = u.id
    -> ORDER BY t.reference_id, ut.role;
+--------------+----------+---------+--------------------+--------------+----------+---------------+
| reference_id | amount   | status  | participant        | phone_number | role     | balance_after |
+--------------+----------+---------+--------------------+--------------+----------+---------------+
| TXN10001     |  5000.00 | SUCCESS | Orla ISHIMWE       | 0781234567   | SENDER   |      14900.00 |
| TXN10001     |  5000.00 | SUCCESS | Benito KABALI      | 0787654321   | RECEIVER |      25000.00 |
| TXN10002     | 12500.00 | SUCCESS | Orla ISHIMWE       | 0781234567   | SENDER   |       2400.00 |
| TXN10002     | 12500.00 | SUCCESS | Kigali Supermarket | 0789998888   | RECEIVER |     150000.00 |
| TXN10004     | 50000.00 | SUCCESS | Charlie HABYARIMANA | 0785554444  | RECEIVER |      52000.00 |
+--------------+----------+---------+--------------------+--------------+----------+---------------+
5 rows in set (0.00 sec)

mysql>
```

3. UPDATE

You can run the update code blocks we have in ***crud_tests.sql*** and get a sample view of **UPDATE** operations on the database.

```
mysql> UPDATE transaction
    -> SET status = 'SUCCESS'
    -> WHERE reference_id = 'TXN10005';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE user
    -> SET account_type = 'MERCHANT'
    -> WHERE phone_number = '0785554444';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
```

4. SECURITY AND UNIQUENESS

You can run the security and uniqueness rules we have in ***crud_tests.sql*** and get a sample view of **our db constraints and rules** on the database.

```
mysql>
mysql> -- CHECK: reject negative amount
mysql> INSERT INTO transaction (reference_id, category_id, amount, fee, status, raw_sms_text, timestamp)
    -> VALUES ('TXN99999', 1, -500.00, 0.00, 'SUCCESS', 'Test negative amount', '2026-05-18 10:00:00');
ERROR 3819 (HY000): Check constraint 'transaction_chk_1' is violated.
mysql>
mysql> -- CHECK: reject negative fee
mysql> INSERT INTO transaction (reference_id, category_id, amount, fee, status, raw_sms_text, timestamp)
    -> VALUES ('TXN99998', 1, 5000.00, -100.00, 'SUCCESS', 'Test negative fee', '2026-05-18 10:00:00');
ERROR 3819 (HY000): Check constraint 'transaction_chk_2' is violated.
mysql>
mysql> -- UNIQUE: duplicate phone number
mysql> INSERT INTO user (name, phone_number, account_type)
    -> VALUES ('Duplicate User', '0781234567', 'STANDARD');
ERROR 1062 (23000): Duplicate entry '0781234567' for key 'user.phone_number'
mysql>
mysql> -- UNIQUE: duplicate reference_id
mysql> INSERT INTO transaction (reference_id, category_id, amount, fee, status, raw_sms_text, timestamp)
    -> VALUES ('TXN10001', 1, 3000.00, 50.00, 'SUCCESS', 'Duplicate ref test', '2026-05-18 10:00:00');
ERROR 1062 (23000): Duplicate entry 'TXN10001' for key 'transaction.reference_id'
mysql>
mysql> -- UNIQUE: duplicate user role per transaction
mysql> INSERT INTO user_transactions (user_id, transaction_id, role, balance_after)
    -> VALUES (1, 1, 'SENDER', 10000.00);
ERROR 1062 (23000): Duplicate entry '1-1-SENDER' for key 'user_transactions.user_id'
mysql>
mysql> -- FOREIGN KEY: invalid category_id
mysql> INSERT INTO transaction (reference_id, category_id, amount, fee, status, raw_sms_text, timestamp)
    -> VALUES ('TXN99997', 999, 5000.00, 0.00, 'SUCCESS', 'Invalid FK test', '2026-05-18 10:00:00');
```